T Level Technical Qualification in Digital Software Development

Mark Scheme

Specimen Assessment Materials

Employer Set Project

General Marking Guidance

- All students must receive the same treatment. Examiners must mark the first student in exactly the same way as they mark the last.
- Mark schemes should be applied positively. Students must be rewarded for what they have shown they can do rather than penalised for omissions.
- Examiners should mark according to the mark scheme not according to their perception of where the grade boundaries may lie.
- All marks on the mark scheme should be used appropriately.
- All the marks on the mark scheme are designed to be awarded. Examiners should always award full marks if deserved. Examiners should also be prepared to award zero marks if the student's response is not rewardable according to the mark scheme.
- Where judgement is required, a mark scheme will provide the principles by which marks will be awarded.
- When examiners are in doubt regarding the application of the mark scheme to a student's response, a senior examiner should be consulted.
- Crossed out work should be marked unless the student has replaced it with an alternative response.
- Accept incorrect/phonetic spelling (as long as the term is recognisable) unless instructed otherwise.

Levels-Based Mark Scheme Guidance

Levels-based mark schemes (LBMS) have been designed to assess students' work holistically. They consist of two parts:

1. **Indicative content**
   Indicative content reflects content-related points that a student might make but is not an exhaustive list. Nor is it a model answer. Students may make some or none of the points included in the indicative content as its purpose is as a guide for the relevance and expectation of the responses. Students must be credited for any appropriate response.
2. **Levels-based descriptors**
   Each level is made up of a number of traits which when combined together articulate the quality of response that a student needs to demonstrate. The traits progress across the levels to demonstrate the different expectations of each level. When using a levels-based mark scheme, the 'best fit' approach should be used.

Applying the levels-based descriptors

Examiners should take a 'best fit' approach to determining the mark.

- Examiners should first make a holistic judgement on which level most closely matches the student's response. Students will be placed in the level that best describes their answer. Answers can display characteristics from more than one level, and where this happens markers must use any additional guidance (e.g. weighting of traits) and their professional judgement to decide which level is most appropriate.
- The mark awarded within the level will be decided based on the quality of the answer and will be modified according to how securely all traits are displayed at that level:
  - Marks will be awarded at the top of that level if the student has evidenced each of the descriptor traits securely.
  - Where the response does not securely meet all traits, the marks should be awarded based on how closely the descriptor has been met.

Task 1 – Planning a project

Indicative content and marker guidance

Gannt chart:

- This is a large project so would expect to see tasks broken down into smaller chunks where sensible, for example:
  - May show use of an Agile approach (or similar)
  - Large modules (e.g. Back-end database module) broken down into multiple sections of development and unit testing
  - Integration testing may be split into smaller chunks and distributed at different times when additional modules are completed
  - It is predicted that there will be three minor issues per module. Testing takes 21 hours (3 days) per module. It would therefore make sense to break each module into three phases, each with a day of testing and 2 days of minor fixes
  - Although major issues can't be predicted completely, it would be logical to assume that these are more likely to happen at certain key points; for example, when integrating and deploying units.
- There should be sensible use of concurrent and serial tasks, for example:
  - Installing server and upgrading infrastructure could be happening while the initial software modules are being developed
  - Many other modules would be dependent on the back-end database so that would need to be developed first
  - Investments and savings is another major module that would provide data for other modules, so should also be developed early in the timeline
  - Integration testing would be expected after a module has been unit tested and before it is deployed
  - Integration testing should only take place when there is at least one other completed module with which to integrate.
- There is no single correct way to organise the plan but task orders should be sensible; for example, 'create a test plan' should occur BEFORE testing commences, acceptance testing would occur much later in the process when the system is nearer completion etc.
- The order and implementation of the project may vary significantly depending on the SDLC approach that students are taking (check against student rationale); for example:
  - for a RAD/Agile that looks at a minimum value product (MVP) they may 'deploy' some of the modules very early on, after a short portion of development time, and then test and develop further, deploying more modules as they go
  - or they may decide that their approach is to have most modules mostly developed and then deployed later.

Costs

Note these are approximate costs and the specific costs in the student's solution would vary depending on the decisions made.

- The cost of the project manager should also be included (approx. £16,000 depending on the length of the project). It would be expected that the project manager would be paid for every day of the project.
- Costs for project staff should be calculated using ONLY the number of days that the work took and NOT the total number of days of the project.
- Students should also include the current ongoing costs (£1,580,900) in their calculations as a starting point for costs – project costs are in addition to the current costs.
- Any loss/profit should be carried forward to the next year.
- This project is easily affordable – the company will still make significant profits in the year of the project (approx. £400,000) and the three-year forecast shows very healthy profit.
- The choice of server makes only minimal impact on the year of the project. The cloud server is slightly cheaper in the year of the project, but more expensive over three years; however, this still makes minimal impact on profits overall.
- Example costings – see attached solution. Note that this is only indicative, and students could solve the problem in a number of ways; credit costings that reflect the students' plan.

Rationale

The rationale will show the reasoning for the chosen project development approach demonstrated in the project plan and points the students may consider, although some of these will vary depending on the choices students make in terms of organisation. These include but are not limited to:

Timings and order of tasks

- Students should identify whether the project deadline is realistic/achievable or not.
- It should be quite comfortable to meet the deadline with sensible concurrent and consecutive tasks. There would, however, be very little slack time. How have they dealt with this risk?
- Although there is little slack time, there are times when some members of staff are not working, so workers could be doubled up on the early units, which could reduce development time.

- Where/when have minor and major faults been allocated/considered? It may make sense in initial planning to align minor faults with testing and major faults when modules are in the process of integration testing and deployment: it is common to discover additional errors at these stages. Students should justify how and why they have organised testing – for example, if all testing is at the very end, why have they chosen to do this and what additional risks does it pose.

Choice of staff and how they have been allocated (e.g. skills, industry expertise, experience)

- Impact of choice of worker on the costs, such as:
  - Sarah O'Toole (SSE) – is very expensive but highly skilled so it would be best to have her working on critical modules and phases (e.g. Investments and savings module, and dealing with major faults)
  - Gina Diaz (DE) – this is a data-heavy project so expect her to be allocated to data-driven modules, e.g. Back-end database, Analytics
  - Ahad Shafiq (JSE) – very inexperienced. It would not be sensible to allocate them to a task on their own. Expect them to be doubled up with another developer – this increases costs but would be sensible to do to reduce risk
  - Terry Duran (JSE) – although a junior software engineer, they are quite experienced and highly skilled so could be used on a range of different tasks
  - Marius Bronski (NE) – should be allocated to hardware-based tasks only (e.g. upgrading infrastructure, installing physical server). May also be allocated to fixing major faults as a contingency as nature of the major fault cannot be predicted and may be hardware-based in nature.
- Consideration of over-allocation/contingency workers, e.g. for major faults it may be sensible to allocate the SSE and the data engineer as the nature of the major error can't be predicted. This increases the planned costs, but these can be recouped if a member of staff isn't needed.
- Have they over-allocated some tasks? E.g. major faults – assigned multiple staff.
- The plan should include an installation/configuration task of either the physical or cloud server (not both). Students should provide a justification as to their choice.

Potential risks

- Only a very limited amount of slack time – 17 weeks is a tight timescale for such a project and may not be feasible; even slight slippage could cause the project to go over deadline.
- Many of the subsequent units rely on the database and the investments and savings units; if these are delayed, then this will impact significantly on the project.
- Junior software engineer (Ahad Shafiq) is very inexperienced so could be an unknown quantity. Likely to need monitoring and support/doubling up with tasks, which:
  - increases costs
  - reduces the number of tasks that can be completed concurrently
  - OR if the JSE is assigned to a task individually that increases the risk that additional errors will occur, they are likely to take longer.
- Discussion of the relevant risks of a purely cloud-deployed system versus the physical particularly in relation to sensitive financial data.

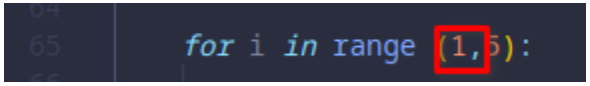
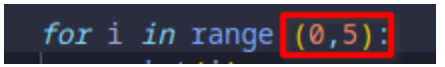
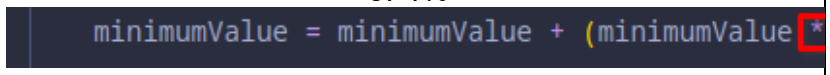
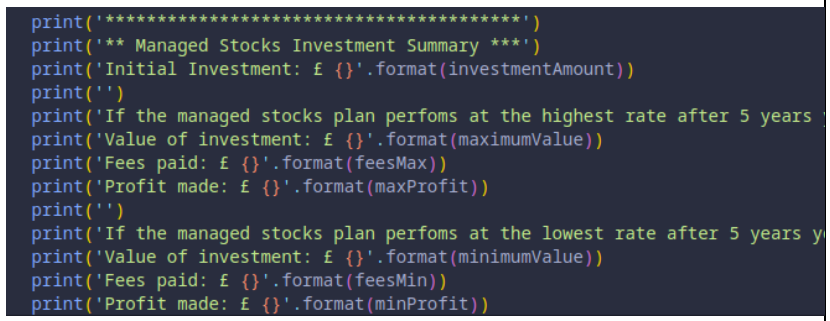
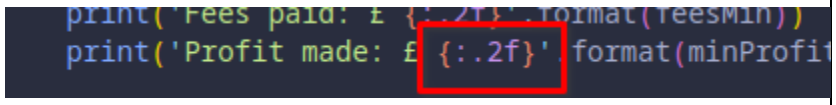
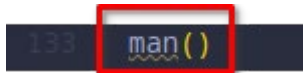
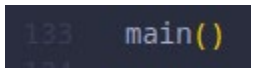
Choice of resources

- A justification of which server option they choose should be presented (e.g. set-up costs versus overall costs over three years, reliability and scalability). Expect consideration of the critical nature of the data, e.g. security concerns of cloud, reliability and redundancy concerns of a single physical server, scalability needs etc.
- Impact of choice of worker on the costs.
- Consideration of over-allocation/contingency workers, e.g. allocating staff with different specialties to ‘Major Faults’ as it is not possible to predict the format that these might take.
- The plan should include an installation/configuration task of either the physical or cloud server (not both). However, if both have been included is there a rationale for this, e.g. back-up/redundancy of data and systems?

Assessment focus	Band 0	Band 1	Band 2	Band 3
	0	**1**	**2**	**3**
Gantt chart	No rewardable material	Project tasks are sometimes organised in logical and efficient manner making some use of an appropriate project management methodology to provide some accurate prediction of the project's timescales.	Project tasks are organised in a mostly logical and efficient manner making use of an appropriate project management methodology to provide mostly accurate predictions of the project's timescales.	Project tasks are organised in a thoroughly logical and efficient manner using an appropriate project management methodology to provide thoroughly accurate predictions of the project's timescales.
	0	**1**	**2**	**3**
Project costings	No rewardable material	Some accurate costs have been added to the plan resulting in an estimate of limited accuracy.	Mostly accurate costs have been added to the plan resulting in an estimate that is largely accurate.	Fully accurate costs have been added to the plan resulting in an accurate estimate.
	0	**1**	**2**	**3**
Resource allocation	No rewardable material	Resources have sometimes been assigned to the project effectively but there are some major and/or significant errors or omissions.	Resources have mostly been assigned to the project effectively, but there are some minor errors or omissions.	Resources have consistently been assigned to the project effectively.

Assessment focus	Band 0	Band 1	Band 2	Band 3
	0	**1–3**	**4–6**	**7–9**
Rationale	No rewardable material	Rationale for project planning decisions demonstrates some effective consideration of: • order and timing of tasks • allocation of team members • potential benefits and risks • impact of decisions on timings and costs.	Rationale for project planning decisions demonstrates mostly effective consideration of: • order and timing of tasks • allocation of team members • potential benefits and risks • impact of decisions on timings and costs.	Rationale for project planning decisions demonstrates a thorough and perceptive consideration of: • order and timing of tasks • allocation of team members • potential benefits and risks • impact of decisions on timings and costs.

Indicative content and marker guidance		
Line number and error category	**Description of error**	**Possible fix**
Line 4 Minor error	Syntax error – missing colon at end of function definition `def get_check_client_ID ()` **Identified through IDE output error/debugging**	`def get_check_client_ID ():`
Line 8 Major error	Logical error in ClientID length check – only flags if more than 10 characters but not if fewer than 10 `if len(clientID) > 10 :` **Identified through test data/debugging and confirmed with testing**	Change relational operator/comparison logic `if len(clientID) != 10 :`
Line 34 Minor error	Amount value not cast/turned as a numerical data type `return amount` **identified through test data/debugging and confirmed with test data**	`return float(amount)`
Line 48 Major error	Logical error – will only allow maximum of 19999 rather than 20000 `if choice == "1" and investmentAmount >= 20000:` **Identified through test data/debugging and confirmed with testing using boundary data**	`if choice == "1" and investmentAmount > 20000:`

Line 65 Major error	Range in for loop incorrect – will only calculate 4 years  **Identified through test data/debugging and confirmed with testing**	 **NOTE** – Student may use an alternative but logically correct fix (e.g. range (1,6)); credit any logically correct response
Line 109 Major error	Calculation error/incorrect value/percentage for lower boundary of interest rates – calculating 40% instead of 4%  Identified through test data/debugging and confirmed with testing	
Lines 118 to 136 Minor error	Values not being output using 2 d.p.  **Identified through debugging/test data**	 **NOTE** – Student may use any alternative option for outputting to 2 d.p. such as round method rather than string-handling functions
Line 128 Minor error	Typo in calling of main program function  **Identified by IDE output error**	

The test plan/log should also include tests that show how a range of aspects of the program have been tested, including how areas of the program that appear to be coded correctly have been tested to ensure outputs are correct and the program is robust. These may include (but are not limited to):

- Different combinations of numbers of value of investment and plan type
- Investment values that sit on the boundary of the maximum allowed, e.g. 19999, 20000, 20001
- Different lengths of clientID number to check logic, e.g. (0, 9, 10, 11).

A limited understanding of program requirements would be typified by only identifying and fixing the minor errors that would be highlighted by the IDE but would not identify and fix significant/major errors that require understanding of logic.

The number of errors identified is not a hurdle between Bands 2 and 3 for the first two assessment foci; the discriminator is the quality and appropriateness of the tests and data selected, and the level of understanding shown of the **process**.

Note – the suggested fixes are for guidance only; credit alternative solutions that are logically correct and would produce the correct/required outcomes.

Assessment focus	Band 0	Band 1	Band 2	Band 3
	0	**1–3**	**4–6**	**7–9**
Use of testing to identify defects	No rewardable material	Tests selected show a basic understanding of the identified program requirements. Test log includes some appropriate tests. Testing has identified some errors in the code provided.	Tests selected show a good understanding of the identified requirements. Test log includes a good range of tests. Testing has identified most errors in the code provided.	Tests selected show a thorough and detailed understanding of the identified program requirements. Test log includes a comprehensive range of tests. Testing has comprehensively identified the errors in the code provided.
	0	**1**	**2**	**3**
Understanding of the testing process	No rewardable material	Test log shows a basic understanding of how errors/problems were identified and how they were rectified.	Test log shows a good understanding of how errors/problems were identified and how they were rectified.	Test log shows a thorough and detailed understanding of how errors/problems were identified and how they were rectified.
	0	**1–3**	**4–6**	**7–9**
The solution	No rewardable material	Code has some functionality, but significant errors still persist. Changes made apply some precise logic and programming structures which would result in some correct outcomes.	Code is mostly functional, but some minor errors still persist. Changes made apply mostly precise logic and programming structures which would result in mostly correct outcomes.	Code is fully functional. Changes to code apply fully precise logic and programming structures throughout which would result in consistently correct outcomes.

Task 3 – Designing a solution

Indicative content and marker guidance

General guidance

- Algorithm designs should demonstrate decomposition of the problem into simpler and more understood primitives.
- The design should provide high level coverage of the process as well as identify reusable components.
- Detailed algorithms (pseudocode) do not need to be provided for ALL repeated processes. For example, if the process for calculating average likes, comments and shares is very similar, the student would not need to provide algorithms for each post type; rather, they should show how reusable code would make appropriate, efficient use of passed variables. The student may provide some additional annotation to explain the process as necessary.
- Good decomposition will show all the necessary processes and sub-processes that make up the main problem.
- A decomposition diagram is not required; decomposition should be demonstrated through detail and process breakdown in the flow charts or pseudocode.
- If pseudocode is used for the algorithms:
  - No specific 'style' is required – allow any sensible method to communicate programmatic structures. Note – owing to the nature of the task, many students may take a very Pythonic approach
  - For 'conventions' look for internal consistency and clarity – e.g. use of capitalisation to identify key command words, use of indentation to show hierarchy etc.

Some general characteristics of a good algorithm that may be demonstrated:

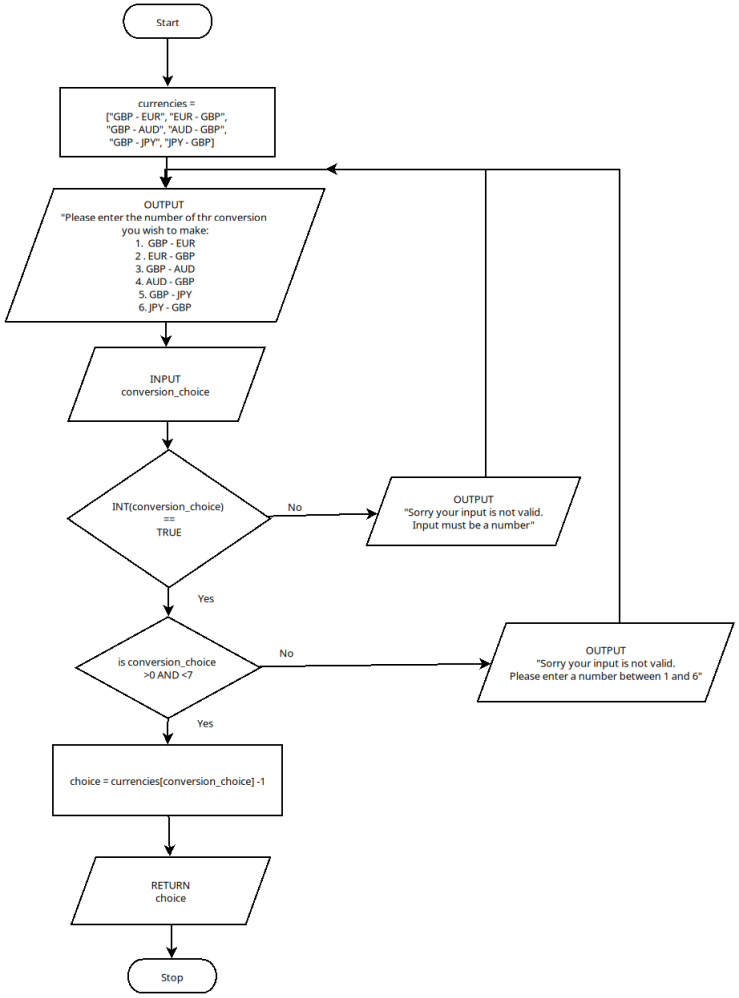
- the steps are clearly defined
- each step is uniquely defined and should depend on the input and the result of the preceding steps
- receives input, selection of type of audience interaction, input coding for usability may also be considered
- produces appropriate type of output (e.g. screen display, return value or return list), which results are required, what happens if no results can be computed (e.g. an error message)
- sensible naming conventions for variables and processes
- use of key words, symbols, hierarchies and structures as appropriate to the chosen method to represent the algorithm (i.e. pseudocode or flow chart).

Scenario-specific characteristics may include:

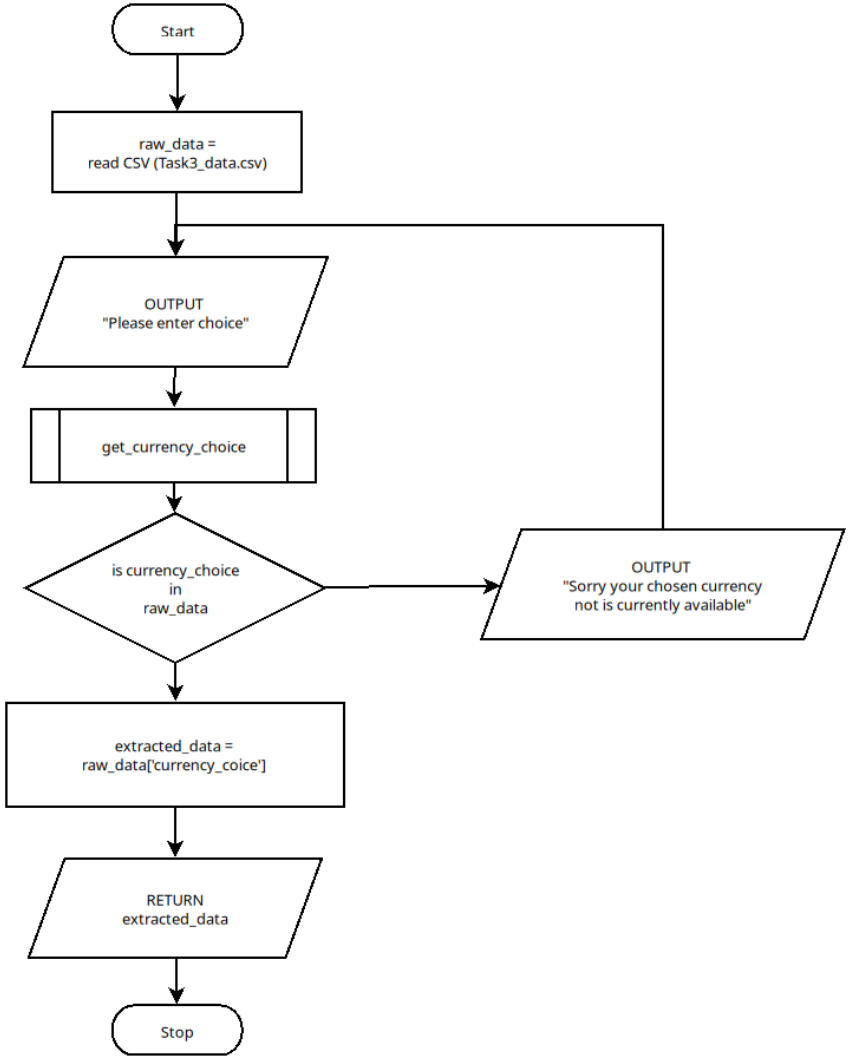
- Links to CSV to get currency figures
- Sensible use of CSV or run-time data structure to hold most up-to-date rates
- Understanding of given data such as:
  - Use of header row in CSV to identify currency conversion, e.g. GBP – EUR
  - Most recent conversion rate is in the final row of the CSV
  - Date column is a string and would need converting to time/date format if it is to be used to ascertain meaningful information
  - Conversion rates are also strings and would need converting to floats
  - Conversion rates use six decimal places. This is good precision when looking at trends but may need rounding to fewer decimal places to be meaningful/sensible for a user
- Options for available currency conversions
- Simplification for user, e.g. choose a number from menu rather than type currency
- Handling of input – conversion from string to number and reduction of potential errors.

Example flow chart algorithms

Function: Get user currency choice



Extract data for requested currency from data source



Example detailed pseudocode algorithms

Note – these are intended to be indicative of the types of algorithms that may be presented. These do not show all processes. Accept any responses that provide logically correct outcomes/solutions based on the given requirements.

Main menu

```
WHILE not_valid_input_flag = TRUE
    SEND 'Please choose an option from the list' TO DISPLAY
    SEND <list of options> TO DISPLAY
    RECEIVE menu_choice (float) FROM KEYBOARD
    IF menu_choice NOT float THEN:
        SEND 'Input must be a numerical value' TO DISPLAY
        SET not_valid_input_flag TO TRUE
    ELSE IF menu_choice <1 OR >6 THEN:
        SEND 'That is not a valid choice' TO DISPLAY
        SET not_valid_input_flag TO TRUE
    ELSE:
        RETURN menu_choice
```

Get latest currency rate

```
IMPORT data-handling library

READ 'Task3a_data.csv'
SET dataframe TO Task3a_data.csv
SET currency TO currencies[menu_choice]
SET conversion_rate to dataframe['currency', -1].
RETURN conversion_rate
```

Conversion

```
WHILE not_valid_input_flag = TRUE
    SEND 'Please enter amount to convert' TO DISPLAY
    RECEIVE conversion_amount (float) FROM KEYBOARD
    IF conversion_amount NOT float THEN:
        SEND 'Input must be a numerical value' TO DISPLAY
        SET not_valid_input_flag TO TRUE
    ELSE IF conversion_amount <1 THEN:
        SEND 'values must be greater than 0' TO DISPLAY
        SET not_valid_input_flag TO TRUE
    ELSE:
        RETURN conversion_amount
SET amount_recieved TO conversion_amount * conversion_rate
SEND 'You will receive <amount_recieved>' TO DISPLAY
```

Assessment focus	Band 0	Band 1	Band 2	Band 3
	0	**1–3**	**4–6**	**7–9**
Decomposition of the problem	No rewardable material	Basic or superficial decomposition of the identified problems that superficially covers the required: • inputs • processes • outputs.	Mostly detailed decomposition of the identified problems that sufficiently covers the required: • inputs • processes • outputs.	Thorough and detailed decomposition of the identified problems that comprehensively covers the required: • inputs • processes • outputs.
	0	**1–2**	**3–4**	**5–6**
Application of logical thinking and conventions	No rewardable material	Algorithms would produce some correct outcomes as a result of: • some precise logic • some appropriate structure and sequence but which is likely to be inefficient. Some use of accepted conventions.	Algorithms would produce mostly correct outcomes as a result of: • mostly precise logic • appropriate structure and sequence but which may lack efficiency. Mostly appropriate use of accepted conventions though some minor inconsistencies may still exist.	Algorithms would produce correct outcomes as a result of: • precise logic • appropriate and efficient structure and sequence. Appropriate and consistent use of accepted conventions.

Assessment focus	Band 0	Band 1	Band 2	Band 3
	0	1	2	3
Communication of the design	No rewardable material	Superficial communication of the design uses technical language which is only sometimes appropriate for the audience.	Adequate communication of the design uses technical language which is mostly appropriate for the audience.	Effective communication of the design uses technical language which is appropriate for the audience.

Task 4a – Developing the solution

Indicative content and marker guidance

The solution

- Provides developed coded solution that utilises the given code and adds the additional functionality as stated in the requirements.
- Integration of exiting code may include:
  - 'import' function to pull code as a whole in when needed (note that the given code and student code will need to be in the same folder)
  - integration into students' own code base as a function.
- The solution will be well structured and modular in nature – clear use of sub-sections; this may be separated modules or the use of procedures, functions or classes.
- Code will be annotated to aid future maintenance of the system.
- Data/information should be output in a meaningful way to the user. This may include use of a data frame, graph and/or text-based summary.
- Output data should show consideration of 'performance over time' – higher level responses will show greater discrimination (e.g. basic level = all data for a single currency against data. Higher level = data extracted for specific timescales (e.g. last 7 days, last 14 days)).
- All variables and structures will be appropriately named.
- Code should be robust; typical errors that may be accounted for in this scenario include negative values, non-numerical characters, entering a choice not provided in the menu.

Possible areas included that contribute to 'user experience':

- Outputs are meaningful and make sense to the user, e.g. value outputs are accompanied by meaningful text to contextualise them.
- Simplification of inputs, e.g. choice of a number instead of typing currency name.
- Numerical values are rounded to a sensible number of decimal places.
- Use of visualisation, e.g. graphs for currency over time.
- Helpful messages and robust input handling.

Security considerations relating to the solutions could include:

- Avoiding global variables, passing data back from functions.
- Avoiding the use of a single generated data frame to ensure security of the data – good practice to generate a new data frame within a function.
- Error handling: if the system crashes, is any data returned in the error message?

Example code snippets for parts of the solution:

Note – these are intended to be indicative of the types of responses that may be presented. These **do not** show all processes. Accept any responses that provide logically correct outcomes/solutions.

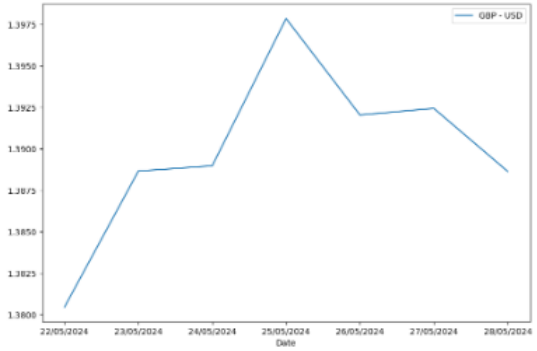
GBP - USD comparison menu with error handling to accept user choice

```
while flag:
    print("#########################################")
    print("Welcome! Please choose an option from the list")
    print("1. Currency Conversion")
    print("2. Value of GBP to USD")
    print("3. Value of USD to GBP")
    print("#########################################")

    main_choice = input("Please enter the number of your choice (1-3): ")

    try:
        int(main_choice)
    except:
        print("Sorry, you did not enter a valid choice")
        flag = True
    else:
        if int(main_choice) < 1 or int(main_choice) > 3:
            print("Sorry, you did not neter a valid choice")
            flag = True
        else:
            return main_choice
```

Use of matplotlib library to generate a graph

```
seven_day_gbp.plot(x="Date", y = "GBP - USD")
plt.show()
```

Code generates a line graph from the extracted dataframe and defines the data for X and Y axis



Use of import to integrate the given code

```
def main_menu_actions():
    if main_choice == 1:
        import Task4a_RBSX_currency_conversion
    elif main_choice == 2:
        gbp_process_data()
```

All menu options called as separate self-contained functions to

Using data structures (dataframes) to extract and display a selection of data and displaying an average

```
def gbp_process_data():

    main_df = pd.read_csv("Task4a_RBSX_data.csv")

    gbp_choice = int(gbp_menu())
    #try to convert date
    if gbp_choice == 1:
        gbp_df = main_df[["Date", "GBP - USD"]]
        seven_day_gbp = gbp_df.iloc[-7:]
        seven_day_avg = round(seven_day_gbp["GBP - USD"].mean(),3)
        seven_day_gbp.plot(x="Date", y = "GBP - USD")
        plt.show()
        print("GBP to USD last 7 days")
        print(seven_day_gbp)
        print("")
        print("The aveage convesion rate for the last 7 days is: ", seven_day_avg )

    elif gbp_choice == 2:
        gbp_df = main_df[["Date", "GBP - USD"]]
        fourteen_day_gbp = gbp_df.iloc[-14:]
        fourteen_day_avg = round(fourteen_day_gbp["GBP - USD"].mean(),3)
        fourteen_day_gbp.plot(x="Date", y = "GBP - USD")
        plt.show()
        print("GBP to USD last 14 days")
        print(fourteen_day_gbp)
        print("")
        print("The aveage convesion rate for the last 14 days is: ", fourteen_day_avg )
```

Code extracts data for different time frames (last 7, 14 and 30 days). Calculates and average an generates a line graph of the data

Note – 30 days not shown on this screen shot

Assessment focus	Band 0	Band 1	Band 2	Band 3	Band 4
	0	**1–2**	**3–4**	**5–6**	
Functionality	No rewardable material	The solution implements code with some functionality but some major errors still persist.	The solution implements mostly functional code but some minor errors still persist.	The solution implements functional and efficient code throughout.	
	0	**1**	**2**	**3**	
Logic and programming structures	No rewardable material	The code uses some precise logic and programming structures which would result in some correct outcomes.	The code uses mostly precise logic and programming structures which would result in sufficiently correct outcomes.	The code uses some precise logic and programming structures which would result in some correct outcomes.	
	0	**1**	**2**	**3**	
Robustness	No rewardable material	The code handles some common user errors.	The code handles most common user errors.	The code thoroughly handles common, and most unexpected, user errors.	

Assessment focus	Band 0	Band 1	Band 2	Band 3	Band 4
	0	**1–2**	**3–4**	**5–6**	
Security	No rewardable material	The code mitigates against some common vulnerabilities as a result of some effective application of secure coding practices.	The code mitigates against most relevant vulnerabilities through mostly effective application of secure coding practices.	The code thoroughly mitigates against relevant vulnerabilities through effective application of secure coding practices.	
		1–2	**3–4**	**5–6**	**7–8**
Code organisation	No rewardable material	The code is partially reusable by a third party but would present significant difficulties through the use of: • inconsistent naming conventions • limited logical organisation • limited informative commenting.	The code is partially reusable by a third party but would present some minor difficulties through the use of: • some consistent naming conventions • some logical organisation • some informative commenting.	The code is reusable by a third party and would present only a few minor difficulties through the use of: • mostly consistent naming conventions • mostly logical organisation • mostly informative commenting.	The code is easily reusable by a third party through the use of: • consistent and appropriate naming conventions • fully logical organisation • highly informative commenting.

Assessment focus	Band 0	Band 1	Band 2	Band 3	Band 4
		1–2	3–4	5–6	7–8
User experience	No rewardable material	Basic user experience is provided through limited effective use of: • input handling • user guidance and error messages • outputs.	Adequate user experience is provided through somewhat effective use of: • input handling • user guidance and error messages • outputs.	Good user experience is provided through mostly effective use of: • input handling • user guidance and error messages • outputs.	Excellent user experience is provided through consistently effective use of: • input handling • user guidance and error messages • outputs.

Task 4b – Reflective evaluation

Indicative content and marker guidance

Indicative content will vary according to the approach students have taken in Task 4a and the effectiveness of the solution they created.

Generic features of effective evaluations are likely to include:

- the extent to which the solution meets the:
  - requirements of the set task brief
    - How did they choose to interpret the 'patterns over time'?
    - How did they compare the two sets of different currencies (e.g. average over time, increases/decreases within specific timeframes)?
  - needs of the users
    - Quality of the final data output, the levels of accuracy that they have achieved (e.g. number of decimal places, rounding up versus truncation)
    - How they handled user input, for example, coding menu options
    - Choice of numerical/textual/graph-based output, how and why these were chosen and how they impact the user
- a justification of how the solution could be further developed/enhanced. Higher attaining students will provide meaningful suggestions for improvement that focus on improving the quality and scope of the code solution, such as additional functionality, or alternative ways to interpret and interrogate the dataset. At the lower level, suggestions are likely to focus on aspects that they did not complete or where parts of their solution do not function
- specific examples from the solution to support points made. For lower attaining students, this is likely to become quite descriptive of what the code does, and will often describe the code function by function, rather than explain why they did something. For higher attaining students, expect the student to identify specific aspects of the code, and justify why it was done in that way and how that relates to the requirements of the brief; for example, why providing common/preset timeframes such as last 7 days, last 14 days etc. might be appropriate and how they coded it in such a way that would be reusable with up-to-date data
- contextualisation of any points made and explaining what they did and justifying why.

Contextualisation for this scenario may include:

- Choice of data output (e.g text, table, graph type) – which is most suitable for data over time and why.
- Rounding versus truncation or reducing number of decimal places for the calculation output.
- How existing code was integrated (e.g. refactoring, storing the code in a different file in the same folder and calling when needed.)
- Choice of libraries/functions to get required data.
- How most recent conversion was established and used (e.g. use of date column versus assuming data would be appended to the end and latest is always the last value).
- How this code could be reused with live data and how they chose to incorporate this, e.g. use of index ranges and assuming/ensuring data is always chronological versus use for datetime function to get specific date ranges.
- Use of variables (global versus local, passing data between functions) and the impact this has on their code/quality of the solution.
- Input error handling – e.g. why they might have excluded text or negative values, menu options etc.

Assessment focus	Band 0	Band 1	Band 2	Band 3
	0	**1–2**	**3–4**	**5–6**
Review of outcomes	No rewardable material	Judgements reached are somewhat supported showing a superficial or basic understanding of how well the solution met the: • system requirements • user requirements.	Judgements reached are mostly well supported showing a good understanding of how well the solution met the: • system requirements • user requirements.	Judgements reached are comprehensively well supported showing a thorough and detailed understanding of how well the solution met the: • system requirements • user requirements.
	0	**1**	**2**	**3**
Future developments	No rewardable material	A superficial or simplistic rationale is provided for what future developments should be implemented.	A good rationale which is reasonably well supported is provided for what future developments should be implemented.	A convincing and well-supported rationale is provided for what future developments should be implemented.

Copyright and Acknowledgements

‘T-LEVELS’ is a registered trademark of the Department for Education.

‘T Level’ is a registered trademark of the Institute for Apprenticeships and Technical Education.

The T Level Technical Qualification is a qualification approved and managed by the Institute for Apprenticeships and Technical Education.

‘Institute for Apprenticeships & Technical Education’ and logo are registered trademarks of the Institute for Apprenticeships and Technical Education.

Pearson Education Limited is authorised by the Institute for Apprenticeships and Technical Education to develop and deliver this Technical Qualification.

Pearson and logo are registered trademarks of Pearson.

Copyright in this document belongs to, and is used under licence from, the Institute for Apprenticeships and Technical Education, © 2025